

机票筛选 Agent · 第二阶段工程架构设计讨论 Prompt

我们现在开始「机票筛选 Agent」项目的第二阶段讨论。

第一阶段已经完成正式收口，并形成《机票筛选 Agent · 第一阶段正式收口 · V0》。第二阶段必须将第一阶段文档视为上游产品与领域契约，不重新无限讨论已经定稿的产品语义。

第二阶段的核心目标是：

将第一阶段已经确定的产品逻辑、Domain Model、Requirement / Patch / Impact / Snapshot / Filter / Ranking / Recommendation 等概念，转换为一个实际可编码、可测试、可部署的 V0 工程架构。

本阶段暂时仍以「中国大陆境内机票 V0 原型」为范围，不提前扩展国际航班复杂业务。

一、第二阶段最终需要回答的问题

经过本阶段讨论后，需要能够明确回答：

1. 整个系统宏观上由哪些层和组件组成？
2. 前端、后端、数据库之外，是否还需要 Agent Orchestrator、LLM Layer、Provider Layer、Cache、Task/Queue、Observability 等独立层？
3. 第一阶段定义的 17 个逻辑模块，在工程上应该如何组织和部署？
4. 哪些模块属于同一个服务，哪些需要保持逻辑独立？
5. 一次用户请求从进入系统到返回推荐结果，完整调用链是什么？
6. Requirement State、Patch、ImpactDecision、Candidate Snapshot 等对象如何在模块间流动？
7. LLM 在整个系统中具体参与哪些步骤？
8. LLM 应采用云端 API、大模型 API、本地模型还是混合方案？
9. LLM 调用如何实现结构化输出、校验、重试、降级和成本控制？
10. 航班数据 Provider 如何接入以及如何设计统一 Adapter？
11. 外部航班数据 API、OTA、航空公司数据源应该怎样与内部 Domain Model 隔离？
12. 哪些数据需要持久化数据库，哪些数据应该进入缓存？
13. Candidate Snapshot 应如何存储、版本化、过期和刷新？
14. 系统是否需要 Redis、消息队列、异步任务或任务编排器？
15. Filter、Ranking、Recommendation、Relaxation 的代码结构应该怎样设计？
16. 系统如何实现 Incremental Recompute / Impact Resolver？
17. 前后端 API 应如何设计？
18. 前端应该如何支持聊天、Requirement 修改、搜索状态、推荐卡片和用户确认？
19. 系统如何处理并发、Timeout、Provider Partial Failure、旧结果覆盖等问题？
20. 如何建立 Logging、Tracing、Metrics、Evaluation 和自动化测试？
21. V0 应该采用单体、模块化单体还是微服务？
22. V0 最终采用什么技术栈？
23. V0 如何部署？
24. Codex 开始开发时应该采用什么 Repository / Package / Module 结构？
25. 整个 V0 应按照什么顺序进行开发？

最终需要形成一套 **MVP Architecture**，作为第三阶段 Codex 实际开发的直接上游输入。

二、第二阶段讨论流程

请不要一开始直接选择具体框架或数据库。

讨论按照以下顺序逐步推进，每一步先充分分析，再形成阶段结论。

Phase 1: 系统边界与工程目标

首先重新确认：

- V0 系统负责什么。
- V0 系统不负责什么。
- 哪些能力属于内部系统。
- 哪些能力依赖外部 Provider。
- 哪些内容属于未来版本。
- 哪些第一阶段契约属于第二阶段不可随意修改的上游约束。

重点确认：

User → Agent → Flight Data → Decision → Recommendation

这条链路中的系统责任边界。

输出：

System Boundary V0

Phase 2: 宏观系统架构

讨论整个系统是否应该简单理解为：

Frontend
Backend
Database

还是需要进一步拆分，例如：

Frontend / Client
API Layer
Application / Orchestrator
Domain Layer
LLM Layer
Flight Provider Layer
Data / Persistence Layer
Cache Layer

Async / Task Layer
Observability
External Services

讨论每一层：

- 为什么存在；
- 负责什么；
- 不负责什么；
- 与哪些模块通信。

同时讨论：

- Modular Monolith
- Service-oriented Architecture
- Microservices

哪个更适合 V0。

本阶段暂时重点确定**逻辑架构**，不要过早陷入具体框架。

输出：

V0 Logical Architecture Diagram

以及：

Layer Responsibility Table

Phase 3: End-to-End Request Lifecycle

选择几个典型场景，从用户输入开始完整跑一遍系统。

至少分析：

Case A: 首次搜索

Natural Language
→ Requirement Parser
→ Normalizer
→ Validator
→ Search Planner
→ Provider
→ Snapshot
→ Normalization
→ Derived Features
→ Filter
→ Ranking
→ Recommendation

→ Explanation

→ Frontend

Case B: Ranking-only Patch

例如：

“还是便宜一点更重要。”

分析为什么：

不 Search

不重新获取 Provider 数据

复用 Snapshot

重新 Ranking → Recommendation → Explanation

Case C: Search-space Patch

例如：

“改成明天从上海飞。”

分析：

Requirement Patch

→ Impact Resolver

→ 新 Search

→ 新 Snapshot

Case D: FILTER_EMPTY

分析：

Filter Empty

→ Relaxation Analyzer

→ 用户确认

→ Patch

→ Incremental Recompute

Case E: Provider Partial Failure

分析系统如何避免把：

Provider Error

误认为：

No Flights

输出：

Phase 4: 核心 Data Contract 与 Schema

把第一阶段领域概念进一步转换为可实现的数据契约。

重点设计：

RequirementState

RequirementConstraint

Preference

RequirementPatch

PatchSet

ImpactDecision

SearchPlan

ProviderQuery

ProviderRawResponse

City

Airport

Carrier

FlightSegment

Itinerary

Offer

CandidateSnapshot

ConstraintEvaluation

DerivedFeature

RankingFeatureContribution

RankingResult

RecommendationItem

RecommendationResult

RelaxationOption

ProviderStatus

WorkflowState

要求明确：

- ID
- Version
- Timestamp
- Provenance
- Missing Value
- Enum
- Relationship
- Dependency

暂时优先形成语言无关的 Schema，再考虑最终代码类型。

输出：

Canonical Data Contract V0

Phase 5: LLM Architecture

这一阶段专门解决 LLM 在项目中的位置。

首先分析：

哪些模块真正需要 LLM。

重点评估：

Requirement Parser
Patch Understanding
Ambiguity Detection
Clarification
Explanation Generator

以及哪些模块原则上不应该依赖 LLM：

Constraint Filter
Ranking Calculation
Impact Matrix

Derived Feature Calculation

Provider Data Normalization 中可以确定处理的部分

为每一种 LLM 调用定义：

Input

Output

JSON Schema

Context

Temperature

Latency Requirement

Reliability Requirement

Fallback

Retry

Validation

然后比较三种方案：

Cloud LLM API

Self-hosted / Local Model

Hybrid Architecture

从以下方面比较：

- 中文理解能力
- Structured Output
- Function Calling
- Reliability
- Latency
- Cost
- Deployment complexity
- GPU requirement
- Privacy
- Scaling
- Model maintenance
- Vendor lock-in

最终决定：

V0 LLM Deployment Strategy

同时设计：

LLM Provider Abstraction

避免业务代码完全绑定某一个模型厂商。

Phase 6: Flight Provider Architecture

讨论实际航班数据如何进入系统。

建立：

Search Planner
→ Provider Adapter
→ External Provider API
→ Raw Response
→ Normalization

研究：

- Provider Adapter Interface
- 多 Provider 并发查询
- Authentication
- Rate Limit
- Timeout
- Retry
- Partial Failure
- Provider Status
- Coverage
- Provenance
- Provider-specific Mapping

同时讨论第一版开发是否应该：

先使用 Mock Provider
→ 再接入真实 Provider

避免外部 API 阻塞整个系统开发。

输出：

Provider Adapter Architecture

Phase 7: Persistence / Database / Cache

逐个判断数据究竟应该：

只存在内存

进入 Cache

进入 Persistent Database

还是同时保存。

重点对象：

Requirement State

Conversation / Session

Patch History

Candidate Snapshot

Provider Raw Data

Normalized Flight Data

Derived Feature

Ranking Result

Recommendation Result

Logs / Evaluation Data

然后选择适合的：

Primary Database

Cache

必要时 Object Storage

讨论：

Candidate Snapshot TTL

Offer Freshness

Cache Key

Invalidation

Versioning

Stale / Expired

输出：

Data Storage Architecture

Phase 8: Impact Resolver 与 Incremental Recompute

把第一阶段 Impact Matrix 转成工程设计。

研究：

Patch

→ ImpactDecision

如何判断：

SEARCH

ENRICH

DERIVED

FILTER

RANK

RECOMMEND

EXPLAIN

哪些层失效。

讨论 V0：

是否只实现明确的 Impact Matrix。

还是需要 Dependency Graph。

建立：

dependency

cache invalidation

snapshot coverage

version consistency

等机制。

输出：

Impact Resolver V0 Design

Phase 9: Filter / Ranking / Recommendation Engine

分别建立确定性模块设计。

Constraint Filter

PASS / FAIL / UNKNOWN

ConstraintEvaluation

Reason Code

Scope

Ranking Engine

Feature Score

Normalization

Preference Importance

Weight Mapping

Feature Contribution

Recommendation Selector

BEST_OVERALL

CHEAPEST

EARLIEST_ARRIVAL

BEST_DEPARTURE_TIME

BEST_AIRPORT_MATCH

FALLBACK

Relaxation Analyzer

分析哪些 Hard Constraint 的最小放宽可以恢复候选。

重点保持：

LLM $\neq$ Ranking Engine。

输出：

Decision Engine Architecture

Phase 10: Orchestrator / Workflow Engine

设计真正控制 Agent 运行流程的组件。

讨论：

READY

NEEDS_CLARIFICATION

REQUIREMENT_CONFLICT

SEARCH_EMPTY

FILTER_EMPTY

DATA_INCOMPLETE

DATA_STALE

PROVIDER_ERROR

这些状态如何转换。

明确：

Orchestrator 负责什么。

Requirement 模块负责什么。

Provider 模块负责什么。

Decision Engine 负责什么。

LLM 负责什么。

避免产生一个“万能 Agent 类”。

输出：

Agent State Machine

Phase 11: Concurrency / Async / Task Architecture

研究哪些步骤：

同步执行

并行执行

异步执行。

重点考虑：

多个 Provider 并发查询

LLM 请求

用户连续输入

取消旧请求

Requirement Version

Snapshot Version

Timeout

Retry

Idempotency

Race Condition

旧版本结果覆盖新状态

判断：

V0 是否真正需要消息队列。

如果需要：

解决什么问题。

如果不需要：

什么时候才引入。

输出：

Phase 12: Backend API Design

设计 Frontend 与 Backend 之间的契约。

例如考虑：

Conversation API

Requirement API

Search API

Recommendation API

Patch API

Refresh API

Provider Status API

讨论：

REST

SSE

WebSocket

Streaming

哪些接口适合哪一种通信方式。

输出：

Backend API Contract

Phase 13: Frontend Architecture

讨论用户真正看到的产品界面。

至少考虑：

Chat Interface

Requirement Summary

Requirement Editing

Clarification

Searching State

Provider Progress

Recommendation Cards

Trade-off Explanation

No Result / Relaxation Options

User Confirmation

Refresh Price

Data Freshness

Error State

讨论前端如何与：

Requirement State

Workflow State

Recommendation Result

对应。

输出：

Frontend Interaction Architecture

Phase 14: Observability / Testing / Evaluation

设计：

Logging

Structured Logs

Tracing

Metrics

Error Tracking

LLM Evaluation

Parser Evaluation

Filter Test

Ranking Test

Provider Contract Test

End-to-End Test

Regression Test

尤其建立第一阶段已经定义的核心测试路径：

首次搜索

Ranking-only Patch

Hard Constraint Patch

Search-space Patch

FILTER_EMPTY + Relaxation

Provider Partial Failure

输出：

Testing & Evaluation Strategy

Phase 15: Security / Privacy / Config

讨论 V0 必要的：

API Key Management

Secrets

Environment Variables

User Data

Logs

Rate Limiting

Input Validation

Provider Credential

LLM Data Exposure

不在 V0 过度设计完整企业级权限系统，但必须避免明显的安全架构问题。

输出：

Security Baseline V0

Phase 16: 具体技术栈选择

只有前面的架构确定后，再开始正式选择技术。

分别决定：

Programming Language

Backend Framework

Agent / Workflow Framework

Schema / Validation Library

Database

Cache

Queue

LLM SDK

Provider HTTP Client

Frontend Framework

Testing Framework

Observability

Deployment

Container

CI/CD

每个技术选择都需要说明：

为什么选。

有哪些替代方案。

为什么暂时不选替代方案。

输出：

V0 Technology Stack Decision

Phase 17: Deployment Architecture

确定 V0 如何运行。

至少讨论：

Local Development

Development Environment

Production-like Environment

Docker

Cloud Deployment

Database Deployment

Cache Deployment

Secrets

Logging

Scaling

Environment Configuration

输出：

Deployment Architecture V0

Phase 18: Repository / Code Architecture

为未来 Codex 开发建立清晰代码结构。

设计类似：

frontend/

backend/

domain/

application/

llm/

providers/

ranking/

filter/

recommendation/

infrastructure/

tests/

具体结构应由前面架构讨论结果决定，而不是提前固定。

同时确定：

Domain 层依赖方向。

Provider 如何通过 Interface 隔离。

LLM 如何通过 Interface 隔离。

数据库如何通过 Repository 隔离。

输出：

Repository Structure V0

Phase 19: MVP 开发顺序

不要一次完成全部系统。

设计逐步可运行的开发路径，例如：

Architecture Skeleton

→ Domain Model

→ Requirement Pipeline

→ Mock Provider

→ Snapshot

→ Filter

→ Ranking

→ Recommendation

→ Patch / Impact

→ LLM

→ Real Provider

→ Frontend

→ Persistence / Cache

→ Testing

→ Deployment

具体顺序需要在讨论后正式确定。

输出：

Implementation Roadmap

三、第二阶段最终交付物

第二阶段结束时，需要形成一份正式的：

《机票筛选 Agent · 第二阶段工程架构设计正式收口 • V0》

至少包含：

1. System Boundary
2. Logical Architecture
3. Component Responsibility
4. End-to-End Sequence
5. Canonical Data Contract
6. LLM Architecture
7. Provider Architecture
8. Persistence / Cache Architecture
9. Impact Resolver Design
10. Decision Engine Design
11. Agent Workflow / State Machine
12. Concurrency Model
13. Backend API
14. Frontend Architecture
15. Testing / Evaluation
16. Observability
17. Security Baseline
18. Technology Stack
19. Deployment Architecture
20. Repository Structure
21. MVP Implementation Roadmap

最终目标是让第三阶段不再讨论：

“系统到底应该怎么设计？”

而开始讨论：

“如何按照已经确定的架构，让 Codex 一步一步实现。”

四、讨论原则

整个第二阶段请遵循以下原则：

第一，不推翻第一阶段已经正式收口的产品语义，除非工程验证发现明确矛盾。

第二，严格区分：

Product / Domain Decision

和

Engineering / Implementation Decision。

第三，不为了“架构完整”而提前引入不必要的复杂度。

始终区分：

未来理想架构

与

V0 最小可运行架构。

第四，对每个技术方案说明：

为什么需要；

解决什么问题；

如果现在不做会怎样；

是否应该进入 V0。

第五，优先保持模块逻辑边界清晰，但不意味着所有逻辑模块都必须变成独立微服务。

第六，LLM 应被视为系统中的一个能力组件，而不是整个系统本身。

第七，确定性逻辑尽量使用代码实现；LLM 主要承担自然语言理解与生成类任务。

第八，所有核心数据与推荐结论尽量具有 provenance 和可测试性。

第九，讨论过程中如果遇到重大架构决策，不要立即跳到结论，应比较不同方案的 Trade-off 后再收口。

第十，每完成一个 Phase，请总结：

已经确定什么

暂未确定什么

为什么

对后续 Phase 有什么影响

然后再进入下一阶段。